一、实验题目

基于 MNIST 手写数字数据集，使用 CNN 完成手写数字识别任务。通过搭建与调试 CNN 模型，深入理解 CNN 的网络构成、各核心组件（卷积层、池化层、全连接层、激活函数）的工作原理与作用，对比分析 CNN 相较于全连接神经网络在图像识别任务中的优势：解决了全连接模型参数爆炸、无法有效捕捉图像空间特征、泛化能力弱的核心问题，最终基于实验结果验证 CNN 在图像特征提取与分类任务上的有效性。

二、实验流程

（一）数据准备

1. 数据加载：

```
raw_train = datasets.MNIST(root='./data', train=True, download=True)
raw_test = datasets.MNIST(root='./data', train=False, download=True)
```

datasets 是 torchvision 库中的一个模块，专门用于提供经典机器学习数据集的下载、存储和访问接口。

2. 标准化：

```
# 第一步：把 0~255 的像素范围缩放到 0.0~1.0
images_scaled = all_images / 255.0

# 第二步：Z-Score 标准化
mean, std = 0.1307, 0.3081
images_normalized = (images_scaled - mean) / std
```

CNN 与 MLP 都依赖梯度下降来训练，而梯度下降对输入数据的尺度极度敏感，因此都需要做标准化。

3. **Reshape**：

CNN 的 Reshape：

Conv2d 要求输入格式为 [N, C, H, W]，需要在第 1 维插入通道数（灰度图为 1 个通道）。

```
images_for_cnn = images.unsqueeze(1)
# [10, 28, 28] -> [10, 1, 28, 28]
print(f"CNN 输入形状: {images_for_cnn.shape}")
# torch.Size([10, 1, 28, 28])
```

MLP 的 Reshape：

Linear 要求输入格式为 [N, Features]，需要把整张图的所有像素拉平成一个一维特征向量，即 28 * 28 = 784 个特征。

```
images_for_mlp = images.view(images.size(0), -1)
# [10, 28, 28] -> [10, 784]
print(f"MLP 输入形状: {images_for_mlp.shape}")
# torch.Size([10, 784])
```

CNN 与 MLP 都需要标准化：这两种模型都依赖梯度下降来训练，而梯度下降对输入数据的尺度极度敏感。

4. **数据集划分与创建加载器：**

```
train_loader = DataLoader(train_dataset_manual, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset_manual, batch_size=1000,
shuffle=False)
```

DataLoader 是 PyTorch 中的数据喂送工具，它接收一个 Dataset 对象，在每次训练迭代时自动将数据切分成固定大小的批次（Batch）、按需随机打乱顺序（shuffle=True），并以可迭代的方式逐批返回给模型。

简洁代码实现：

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_data = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_data = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)
```

Compose 本身只接收一个参数：一个变换操作的列表，按顺序依次对每张图片执行。transforms.ToTensor() 将数组转为 Tensor，并将像素值从 [0, 255] 整数缩放到 [0.0, 1.0] 浮点数；transforms.Normalize(mean, std) 执行标准化。上述代码实现数据下载、变换并划分为训练集与非训练集。

（二）实例化 CNN 模型

1. **模型定义**：每个卷积块遵循固定模式：Conv → ReLU → Conv → ReLU → MaxPool → Dropout。

```
# 第一层卷积块
self.conv1 = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=5, padding=2),
    nn.ReLU(),
    nn.Conv2d(32, 32, kernel_size=5, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Dropout(0.25)
)

# 第二层卷积块
self.conv2 = nn.Sequential(
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Dropout(0.25)
)
```

池化层：之所以用两层卷积再接一个池化，是因为单层卷积感受野有限，叠加两层卷积后，第二层的每个神经元实际上能看到输入图中更大范围的像素区域，从而提取更复杂的特征（如边缘→角点→笔画轮廓）。

padding：padding=2 可使 kernel_size=5 的卷积输出与输入保持相同的空间尺寸（same padding），计算公式为：输出尺寸 = (W - K + 2P) / S + 1 = (28 - 5 + 4) / 1 + 1 = 28。

通道数：conv1 第一层：输入 1 通道 → 输出 32 通道，从原始灰度图中提取 32 种初级特征（横线、竖线、斜线等）。conv1 第二层：输入 32 通道 → 输出 32 通道，在这 32 种特征基础上做深化。conv2 第一层：输入 32 通道 → 输出 64 通道，通道数翻倍，提取更抽象的高级特征（笔画组合、局部形状）。conv2 第二层：输入 64 通道 → 输出 64 通道，继续深化。

空间尺寸变化：输入 [N, 1, 28, 28]，经过 conv1（两层卷积，padding 保持尺寸不变）后为 [N, 32, 28, 28]；经过 MaxPool2d(2) 尺寸减半为 [N, 32, 14, 14]；经过 conv2（两层卷积，padding 保持尺寸不变）后为 [N, 64, 14, 14]；经过 MaxPool2d(2, stride=2) 尺寸再减半为 [N, 64, 7, 7]；经过 Flatten 得到 [N, 64×7×7] = [N, 3136]。

全连接层设计：

```
self.fc = nn.Sequential(
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 10)
)
```

将卷积提取的空间特征图展平后，通过两层全连接逐步将 3136 维高维特征压缩到最终的 10 个类别得分。最终输出的 10 个数字是每个类别的原始得分（logit），由 CrossEntropyLoss 在计算损失时内部自动转为概率。

forward() 的数据维度变化：输入 x 为 [N, 1, 28, 28]，经过 conv1 后为 [N, 32, 14, 14]，经过 conv2 后为 [N, 64, 7, 7]，经过 fc 后为 [N, 10]，最终输出 10 个类别的得分。

1.2 优化器与损失函数

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

优化器作用：决定每次反向传播后，权重更新规则。损失函数作用：衡量模型预测结果与真实标签之间的差距，差距越小损失越低。

1.3 模型训练

重复 N 个 epoch：a. 训练阶段，用训练集更新权重；b. 验证阶段，用测试集评估当前效果（不更新权重）。

训练阶段：

```
optimizer.zero_grad()    # 步骤1：清空上一个 batch 的梯度残留
output = model(data)    # 步骤2：前向传播，得到预测得分
loss = criterion(...)   # 步骤3：计算预测值与真实标签的差距（损失）
loss.backward()         # 步骤4：反向传播，计算每个参数对损失的梯度
optimizer.step()        # 步骤5：根据梯度更新所有参数（权重）
train_loss += loss.item()  # 步骤6：累加损失，用于打印平均损失
```

验证阶段：

```
model.eval()                # 关闭 Dropout，让模型进入推理模式
with torch.no_grad():       # 关闭梯度计算，节省显存和计算
    pred = output.argmax(dim=1)    # 取10个得分中最大值的索引，即预测的数字
    val_acc += pred.eq(target...) # 统计预测正确的数量
val_acc /= len(dataset)     # 正确数 / 总数 = 准确率
```

三、结果分析

（一）训练与验证曲线

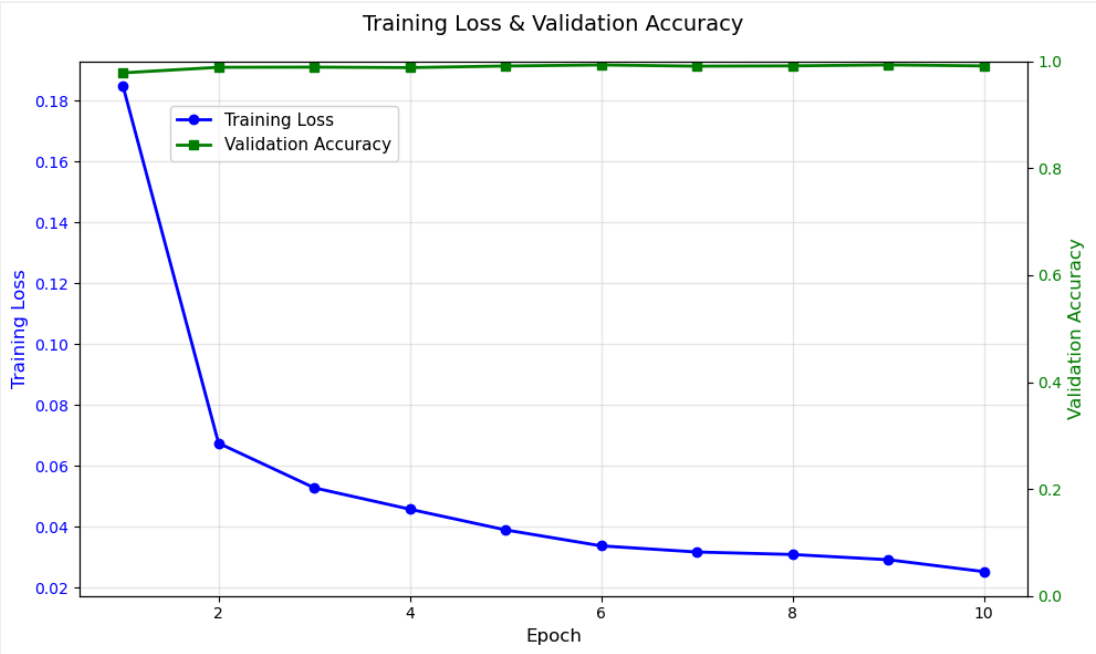

图1：训练损失与验证准确率曲线

模型在验证数据集上达到近 99%（98.7% 以上）的准确率，经过 10 个 epoch。在训练过程中，验证准确率几乎每次都高于训练准确率，说明模型没有过拟合训练集。

（二）混淆矩阵

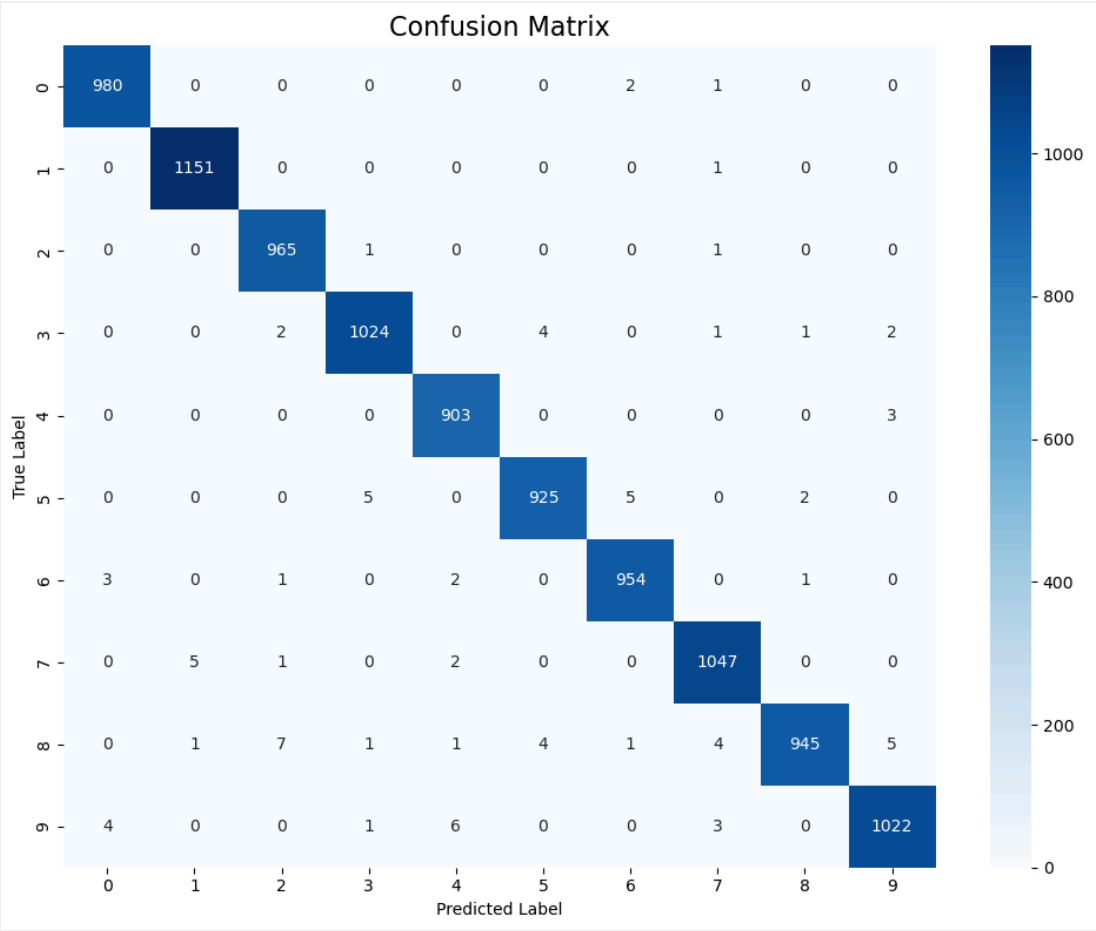

图2：混淆矩阵

CNN 在所有数字上都有很高的准确率，错误很少。但是在处理 7 这个数字时容易被错误地分类为 1。说明当 7 书写的曲线平滑时，1 和 7 之间的特征具有一定的相似性。

四、总结

Q1：为什么需要 CNN

MLP 虽然经过数据标准化与梯度下降优化，理论上也能进行分类，但其忽视了图像的空间结构信息。当 MLP 将 28×28 像素的图像展平为 784 维向量时，相邻像素之间的邻域关系（如笔画的连续性、边缘特征的连贯性）丢失，每个像素被当作独立特征处理。卷积神经网络通过卷积核在局部区域内进行特征提取，保留了像素的空间邻域关系，能够高效地学习从低层笔画边缘、到中层笔画组合、再到高层数字轮廓的分层特征。用更少的参数实现更高的准确率，是处理图像数据的最优选择。

Q2：CNN 模型组成部分

第一部分是卷积块，包含两个卷积块，每块由两层卷积、ReLU 激活、最大池化和 Dropout 组成，通过金字塔式压缩将 [1,28,28] 的输入最终变为 [64,7,7] 的特征，从初级特征逐步迭代到复杂的抽象特征。第二部分是展平层，将 4D 张量转换为一维向量 [3136]，实现从空间特征表示向全连接层的过渡。第三部分是全连接层，由两层线性层组成，将 3136 维向量逐步压缩到 10 维，对应 10 个数字类别的最终输出。

Q3：CNN 模型为什么需要全连接层

卷积层与池化层的职责是特征提取，而全连接层的职责是决策。卷积层学到了图像中最具判别性的特征模式，但这些特征本身仍是无序的原始材料。全连接层接收这些高级特征，通过非线性变换学习特征之间的复杂交互关系，最终映射到各类别的概率得分。没有全连接层，模型就只能提取特征却无法做出最终决策，因此全连接层是作用从理解图像内容到做出分类判断。